Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 15.11.2023.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
   a. Vodite računa da imate sve spremno što vam je neophodno.
   b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2024_Test1_GrupaH_IT1g2023**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Merge Sort*

Napisati metodu ***merge*** - ***Merge Sort*** algoritma za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalazi klasa ***RandomNumberGenerator*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- U okviru klase ***MergeSort*** napisati implementaciju za metodu ***private static void merge***(long[] arr, long[] tmpArray, int lowerIndex, int middleIndex, int upperIndex)
- Obezbediti kod u **Main** metodi za testiranje algoritma.

Zadatak 2 – *Promena redosleda n elemenata reda, koristeći stek*

Implementirati funkciju koja za zadati **red** celih brojeva obrće redosled samo prvih n elemenata reda, koristeći pomoćni **stek**. U okviru tih prvih n elemenata, za svaki od njih koji je paran važiće da se njegova vrednost povećava za 2. Nakon toga, preostali elementi reda ostaju u istom redosledu.

Specifikacija funkcije:

- Funkcija prima broj n, gde je n broj elemenata koje treba obrnuti.
- Funkcija treba koristiti stek (***stack***) kao pomoćnu strukturu.
- Elementi posle prvih n treba da ostanu u originalnom redosledu.

Dodatni zahtevi za rešavanje zadatka:

- Algoritam pisati u okviru klase ***Queue.***
- Zahtevan potpis metode: ***public void ReverseTopNQueueElements (int n)***

Primer1:

- Originalni red: 1 2 3 4 5 6 7 , n = 4

  Među prva 4 elementa, 2 i 4 jesu prani brojevi, te se njihova vrednost povećava za dva. Zato u ispisu 4 postaje 6, a 2 postaje 4.

- Red nakon obrtanja poslednjih 4 elementa: 6 3 4 1 5 6 7

Primer2:

- Originalni red: 1 2 3 4 5 6 7 , n=7
- Red nakon obrtanja poslednjih 7 elementa: 7 8 5 6 3 4 1

Primer3:

- Originalni red: 1 2 3 4 5 6 7 , n=8
- Nevažeća vrednost za n.

Zadatak 3 – *Najduži segment sa rastućim i opadajućim vrednostima*

Čvorovi u povezanoj listi mogu formirati **segment sa rastućim i opadajućim vrednostima** ako zadovoljavaju sledeće uslove:

1. Čvorovi unutar segmenta prvo imaju strogo rastuće vrednosti, a zatim strogo opadajuće vrednosti.
2. Segment mora imati najmanje 3 čvora: najmanje 2 u rastućem delu i 1 u opadajućem delu, ili obrnuto.

Zadatak je da se pronađe i vrati:

1. **Dužinu najdužeg segmenta sa rastućim i opadajućim vrednostima.**
2. **Početnu vrednost segmenta.**
3. **Krajnju vrednost segmenta.**

Ako nijedan takav segment ne postoji, treba vratiti niz [-1, -1, -1].

Dodatni zahtevi:

- Rešavaj ovaj zadatak pomoću strukture liste (*LinkedList*) iz šablona zadatka.
- Implementiraj ga u okviru metode: ***public int[] findLongestSegment()***

Primer1:

- Input 1: lista: 1 -> 2 -> 3 -> 2 -> 1
- Output 1: [5, 1, 1]

Objašnjenje:

Najduži segment sa rastućim i opadajućim vrednostima ima dužinu 5 (1 -> 2 -> 3 -> 2 -> 1), počinje od 1 i završava se na 1.

Primer2:

- Input 2: lista: 5 -> 4 -> 3 -> 2 -> 1
- Output 2: [-1, -1, -1]

Objašnjenje:

Nema rastućeg dela u listi, pa ne postoji validan segment.